8. Visual C#.NET’te Sınıflar

- Bu bölümde;

  - Constructor ve Destructor Oluşturmak
  - Kullanılan Komutlar
  - Kalıtım

  Konularına değinilecektir ve Program yazarken kullanacağımız ve bize büyük kazançlar sağlayan sınıflar hakkında bilgi sahibi olacak ve kullanma yetisi kazanacaksınız.

8.1. Constructor ve Destructor Oluşturmak

- Bir sınıf oluşturulduğu anda çalıştıracağınız kodlar varsa Constructor oluşturmanız gerekmektedir. Yani kullanıcı programı çalıştırdığı anda müdahale edemeden bir kodun çalışmasını istiyorsanız bu constructor oluşturmanız gerekmektedir.
- Bir sınıf bellekten atılırken destructor oluşturulmalıdır. Bir bakıma sınıfı kapatma işlemi gibi düşünülebilir.

- namespace sinif
  {
  class sinifadi
  {
  public sinifadi() constructor
  {
  MessageBox.Show(“Sınıf çalıştırıldı”);
  }
  }
  ~sinifadi() destructor
  {
  MessageBox.Show(“Sınıf bellekten atılıyor”);
  }
  }
- private void Form2_Load(object sender, eventArgs e)
  {
  sinifadi deneme=new sinifadi(); /*Sınıftan nesne oluşturuluyor. constructor işletilir*/
  this.Close(); //destructor işletilir
  }

Public sinifadi deneme =new sinifadi();

- Yordamların dışında bir yere bu şekilde yazılarak oluşturulan nesne tüm proje için geçerlidir.
- Burada kullanılan diğer bir komut olan new komutu sınıftan oluşturulan nesnelere için bellekte yer açmak amaçlı kullanılan bir komuttur.
- Bir metodun public olarak deklare edilmesi başka bir sınıfın nesnesiyle bu metoda ulaşabilmesini sağlar.
- Bir metodun private olarak deklare edilmesi bu metodun sadece bu sınıfta kullanılmasını sağlamaktadır.
- Bir metodun protected olarak deklare edilmesi genelde miras veren sınıflarda kullanılır. Böylece bir metodun sadece miras verdiği sınıflarda kullanılabilmesi sağlanabilir

8.2. Kullanılan Komutlar

- Burada fazladan bir komut kullanılmıyor. Ancak tanımlamaların nasıl yapılacağına değinilecektir.

```
sinifadi deneme=new sinifadi();
```

İle var olan bir sınıftan nesne oluşturulmaktadır. Sınıfların içerisinde oluşturduğumuz fonksiyonlara ve değişkenlere ulaşabilmek ve kullanabilmek için muhakkak sınıftan nesne oluşturmamız gerekmektedir.Yukarıda yazılan kodta nesne sadece bir yordam için geçerlidir. Dolayısıyla oluşturduğumuz nesnenin tüm projede geçerli olmasını istiyorsak yordamların dışında bir yere aşağıdaki şekilde kod yazılır.

8.3. Kalıtım

- Miras alma olacakta bilinir ve çok önemlidir. Yani sınıfta oluşturulan bir değeri veya fonksiyonda bulunan değişkenleri bir başka sınıf tarafından miras alınarak kullanılması olayına denir. Bu sayede kod karmaşası ve zaman kaybı önlenecektir.

Örnek

- namespace sinif
  {
  class sinifmirasveren
  {
  public void mesaj()
  {
  MessageBox.Show(“miras alındı”);
  }
  }
- //
  class sinifmirasalan:sinifmirasveren
  {

  }
  }
- private void Form2_Load(object sender, eventArgs e)
  {
  sinifmirasalan deneme=new sinifmirasalan(); Sınıftan nesne oluşturuluyor.
  deneme.mesaj(); miras yoluyla başka bir sınıftaki fonksiyon kullanılıyor.
  }
- Örnekte görüldüğü gibi sinifmirasalan sınıfı sinifmirasveren sınıfını kullanarak miras almakta ve ona ait bir fonksiyonu kullanmaktadır.

- **Visual C#.NET’te Sınıflar** bölümünü tamamladınız.
- Bu bölümde;
- Sınıfların oluşturulması ve kullanılması, kalıtım olayının kullanılması gibi önemli konuları kullanma yetisi kazandınız.

Visual C#.NET’te SQL Server

- Bu bölümde;
  - Veri Tabanına Erişim
  - Kullanılan Kontroller
  - Kullanılan Nesneler
- Konularına değinilecektir.
- Ayrıca yine bu bölümde;
  - C#.NET ile SQL server arasındaki bağlantının nasıl yapılacağını ve veri tabanına ile iletişimde kullanacağınız bazı komutları öğreneceksiniz.

Veri Tabanına Erişim

- Program yazarken en büyük problemlerden bir tanesi veri tabanına erişimdir. Kod ile veya sihirbazlar kullanılarak veri tabanına bağlanılabilmektedir. Kod ile erişim aşağıdaki örnekte verilmiştir. Kod ile bağlantıda kod yazım ekranın en tepesine aşağıdaki kod eklenir.

```
using System.Data.SqlClient;
```

Örnek

```
private void Form2_Load(object sender, eventArgs e)
{
    SqlConnection baglanti = new SqlConnection("Data Source=localhost;

    Initial Catalog=EBOLUM;

    Integrated Security=True");

    baglanti.Open();
}
```

Yukarıdaki kod ile SQL server‘ın nerde olduğu ve nasıl bağlanılacağı belirtilmiştir.

- Data Source ile Sql server’ın bulunduğu billgisayarın adı.
  Initial Catalog ile veritabanının adı.
  Integrated Security= Windows kimlik bilgileri verilmektedir.
- Yukarıdaki bağlanış Windows authentication ile yapılmaktadır. Bir diğer bağlanma şekli çok gerekli olacak ve kullanmak zorunda kalacağımız SQL server authentication ile yapılandır.

Kullanılan Komutlar

- Burada sql server ile iletişimi sağlayacak komutlar gösterilecektir. Bir önceki dersteki bağlantı aynı şekilde yapılmış olduğunu varsayalım.
- **Örnek:**

```
private void Form2_Load(object sender, eventArgs e)
{
SqlConnection baglanti = new SqlConnection("Data Source=localhost;
Initial Catalog=EBOLUM;Integrated Security=True");

SqlCommand komut = new SqlCommand("INSERT INTO
tabloadi(ad,soyad,adres) values('“TextBox1.Text+"','"+ TextBox2.Text
+"', '"+ TextBox3.Text +"')", baglanti);
komut.Connection.Open();
komut.ExecuteNonQuery();
komut.Connection.Close();
}
```

- Yukarıdaki işlemde veri tabanı ile bir bağlantı oluşturulmuştur. Insert into komutuyla veritabanına kayıt eklenmektedir.
- Tablo adi veritabanındaki oluşturulmuş tablonun adıdır ve içindeki değerlerde alan adlarıdır.
- Values ile istediğimiz değerleri bu alan adlarına sırasıyla göndermekteyiz.
  komut.Connection.Open(); ile bağlantıyı açarız ve komut.ExecuteNonQuery(); ile yazdığımız kodların çalışmasını sağlarız ve close ile bağlantıyı kapatırız.
- Komutlar hep böyle olduğu için bundan sonraki komutlarda sadece insert into ile başlayan kısımlar verilecektir.

- Burada var olan bir kaydı silmek için aşağıdaki kod yazılır. Aşağıdaki kod İnsert into kodunun yerine yazılması gereklidir. Where komutuyla ad alanında ahmet ismine eşit olan tüm kayıtları buluyoruz ve delete komutuyla bunları siliyoruz.

  DELETE FROM tabloadi WHERE ad='ahmet';

- Tabloda var olan bir kaydı değiştirmek istiyorsak update komutu kullanılır.

  UPDATE tabloadi SET ad = 'Bilişim', soyad='enstitüsü' WHERE ad = 'ahmet'

- Yukarıdaki kod tabloadi ismindeki tablodan ad kısmı ahmet ismine eşit olan kayıtlarda ad alanını Bilişim, soyad alanını da enstitüsü yap anlamına gelmektedir.

Bu SQL cümleciklerinde kullanılabilecek bazı diğer komutlar aşağıda verilmiştir.

Between

Select * from tabloadi where maas between 10000 and 1000000

Maas alanı olduğunu varsayarsak tabloda yukarıdaki kod ile 10000 ile1000000 arasında maaşa sahip olan tüm kayıtları göster anlamına gelmektedir.

Like

Select * from tabloadi where ad like ’ %y% ’

Tabloda ad alanında **y** karakteri içeren tüm kayıtları göster anlamına gelmektedir.

ORDER BY

Select * from tabloadi order by ad ASC

Tablodaki tüm kayıtları ad alanına göre küçükten büyüğe sırala anlamına gelmektedir. ASC yerine DESC yazılırsa büyükten küçüğe sıralama yapar.

Count

Select count(ad) from tabloadi where ad='ahmet'

Tabloda ad alanında ahmet ismine eşit kaç tane kayıt varsa sayısal olarak değerini dönderir.

- **Visual C#.NET’te SQL Server** bölümünü tamamladınız.
- Bu bölümde;
- C#.NET ile veri tabanına sihirbaz kullanarak ve kod kullanarak nasıl bağlanıldığını öğrendiniz ve veritabanı ile işlem yapabilme becerisi kazandınız.